Advanced Algorithms and Applications
Assignment 5

1.Solution to Red-Black Tree Violation

Identification of Violation

Upon inserting a new node into a red-black tree, we encounter a violation of the properties of red-black trees. Specifically, a newly inserted node z creates a situation where a red node has a red child, which is not allowed.

Case Analysis

- In the first tree, node z is red and its parent node is also red, which is a direct violation of the red-black tree property that red nodes cannot have red children (Case 1).
- The second tree presents the same issue with node z being red and having a red parent (Case 1).

Fixing the Violations

To fix the red-black tree after the insertion of z, we must follow these steps:

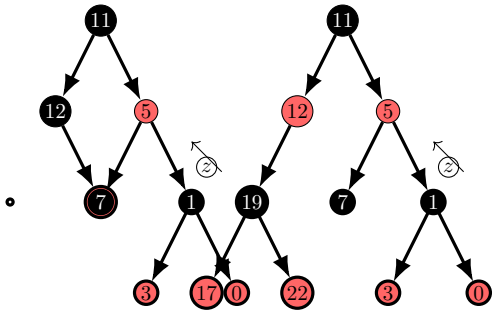
First Tree

1. Recolor the parent of z and its sibling to black.
2. Recolor the grandparent of z to red.
3. Since the grandparent is now red and its parent is black, no further action is necessary as the properties are restored.

Second Tree

1. Recolor the parent of z and its sibling to black.
2. Recolor the grandparent of z to red.

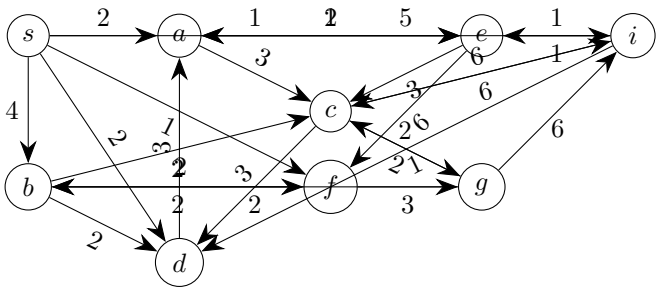
3. Check if the grandparent node's parent is red, which may introduce a new violation up the tree. If this is the case, further rotations or recoloring may be necessary.



2. Adjacency matrix for an undirected graph

Graph

Graph from Adjacency Matrix



Graph Traversals from Node 's'

The graph's traversal sequence for both the Breadth-First and Depth-First Search algorithms, starting from the node labeled 's', is derived from the adjacency matrix. We adhere to alphabetical ordering for the queue (BFS) and stack (DFS).

Breadth-First Search (BFS) Order

The BFS algorithm explores the graph layer by layer from the start node. The sequence is as follows:

- Initiate from 's', enqueue its direct neighbors 'a', 'b', 'd', 'e', 'f'.

- Dequeue 'a', visit and enqueue its neighbor 'c'.
- Continue with 'b', with no new neighbors to enqueue.
- Visit 'd', which leads to 'g' being enqueued.
- Visit 'e', leading to 'i' being enqueued.
- Visit 'f', with no new neighbors to enqueue.
- Proceed to 'c', which has no new neighbors.
- Finally, visit 'g' and 'i', both with no new neighbors.

Hence, the BFS traversal is: $s \to a \to b \to c \to e \to d \to i \to f \to g$.

Depth-First Search (DFS) Order

DFS explores as far as possible along each branch before backtracking. Here's the order:

- Start at 's', push and visit 'a', then 'c', diving deeper.
- From 'c', traverse to 'd', then 'g', going as deep as possible.
- Backtrack to 'd' then to 'a', and now to 'e', exploring new paths.
- Dive from 'e' to 'f', then to 'b', exploring all new nodes.
- Lastly, visit 'i' from 'e', which is the only unvisited node left.

Thus, the DFS path is: $s \to a \to d \to c \to i \to e \to f \to b \to g$.

Note: The actual order may vary based on specific implementation details of the BFS and DFS algorithms.

Construction of a Minimum Spanning Tree

We apply Prim's algorithm to the given graph to develop an MST, which includes all the nodes with the minimum sum of edge weights. The process involves adding the least weight edge at each step.

Algorithm Application

Starting from the designated initial node 's', we perform the following operations to build the MST:

1. Incorporate node 's' into the MST.
2. Select the least weight edge leading from 's', which is to 'e' (weight 1).

3. Add the edge 's' to 'f' (weight 1), noting that multiple edges of the smallest weight may be chosen arbitrarily.
4. Connect 'f' to 'b' (weight 2), as the next smallest connection.
5. Incorporate 'b' to 'd' (weight 2), continuing the sequence of minimal edges.
6. Proceed by connecting 'd' to 'g' (weight 1), the smallest edge available.
7. Connect 'g' to 'c' (weight 2), maintaining the growth of the MST.
8. Include 'c' to 'a' (weight 3), completing the connections to all nodes.
9. Lastly, 'e' to 'i' (weight 1) is added, finalizing the MST.

Completed MST

The constructed MST is composed of the edges:

- (s, e) with weight 1
- (s, f) with weight 1
- (f, b) with weight 2
- (b, d) with weight 2
- (d, g) with weight 1
- (g, c) with weight 2
- (c, a) with weight 3
- (e, i) with weight 1

The aggregate weight of the MST edges is $1 + 1 + 2 + 2 + 1 + 2 + 3 + 1 = 13$.

Dijkstra's Algorithm Shortest Path Solution

Applying Dijkstra's algorithm from the source node 's' to determine the shortest path to all other nodes in the given graph.

Initialization

All nodes are initially marked with a distance of infinity, except for the start node 's', which is marked as 0. The previous node for each node is undefined.

Node	Tentative Distance	Previous Node
s	0	-
a	∞	-
b	∞	-
c	∞	-
d	∞	-
e	∞	-
f	∞	-
g	∞	-
i	∞	-

Algorithm Execution

The algorithm proceeds with the following steps:

- Update the neighbors of 's': a (2), b (4), c (2), d (2), e (1), f (1).
- Select the node with the smallest distance not yet visited, 'e' (1), and update its neighbors.
- Select the next node, 'f' (1), and update its neighbors.
- Proceed by selecting 'a' (2) and then 'c' (2), updating their neighbors each time.
- Continue with 'd' (2), 'b' (3), 'g' (3), and finally 'i' (2).

Final Shortest Paths

The shortest paths from 's' to each node, along with their total distances, are:

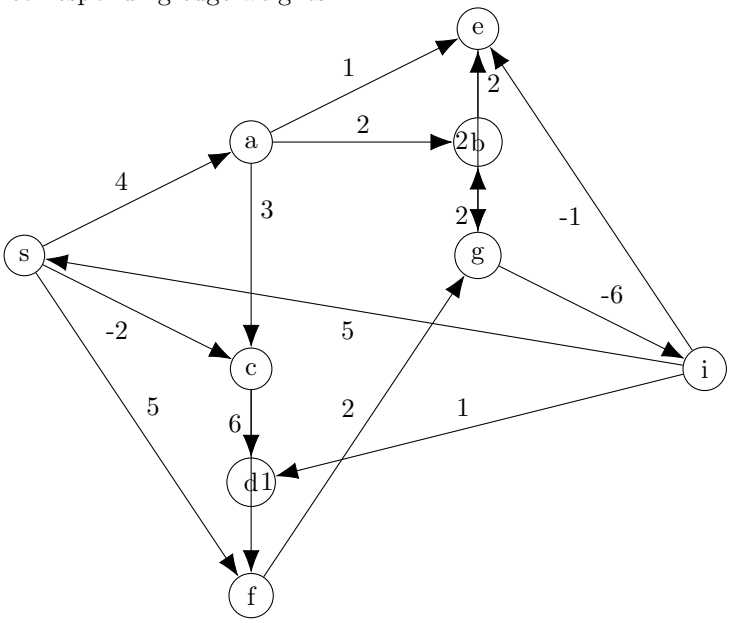
Node	Shortest Path and Total Distance
a	s $\rightarrow$ e $\rightarrow$ a, Distance = 2
b	s $\rightarrow$ f $\rightarrow$ b, Distance = 3
c	s $\rightarrow$ e $\rightarrow$ a $\rightarrow$ c, Distance = 4
d	s $\rightarrow$ f $\rightarrow$ b $\rightarrow$ d, Distance = 5
e	s $\rightarrow$ e, Distance = 1
f	s $\rightarrow$ f, Distance = 1
g	s $\rightarrow$ d $\rightarrow$ g, Distance = 3
i	s $\rightarrow$ e $\rightarrow$ i, Distance = 2

The complete solution provides the shortest paths from the source node to all others in the graph, with the respective distances.

3. BFS and DFS Traversal Orders

Graphical Representation of Directed Graph

The directed graph as defined by the adjacency matrix is depicted below with the corresponding edge weights.



Graph Traversal Sequences

Utilizing the adjacency matrix, we determine the sequence of nodes visited in both Breadth-First Search (BFS) and Depth-First Search (DFS) from the starting node 's'. We will explore all reachable nodes, ignoring the edge weights as they do not influence the traversal order.

Breadth-First Search (BFS) Sequence

BFS explores each level of the graph before moving to the next, ensuring the shortest path in terms of the number of edges from the start node to every other node is explored first.

- Initiate at node 's', visiting it first.
- Proceed to visit the immediate neighbors: 'a', 'c', then 'f'.
- Next, from 'a', continue to unvisited nodes 'b' and 'e'.
- From 'c', move to node 'd'.

- Node 'f' has no new neighbors, so we then move from 'e' to 'g'.
- Lastly, from 'g', visit node 'i'.

Thus, the BFS sequence is: $s, a, c, f, b, e, d, g, i$.

Depth-First Search (DFS) Sequence

DFS delves into the graph as far as possible before backtracking, ensuring a complete path is traced before moving to the next.

- Start at 's', exploring as deep as possible to 'a'.
- From 'a', proceed to 'b', then backtrack to 'a' and go to 'c'.
- Visit 'd' from 'c', then backtrack to 'c' and move to 'f'.
- From 'f', go to 'g', then to 'i', which leads back to 's'.
- Since 'e' remains unvisited from 's', it is visited last.

Consequently, the DFS sequence is: $s, a, b, c, d, f, g, i, e$.

Bellman-Ford Algorithm Application

Given a directed graph with potentially negative edge weights, the Bellman-Ford algorithm finds the shortest path from a single source to all other vertices and detects any negative weight cycles.

Initialization

Distance to source vertex 's' is set to 0. All other vertices are set to infinity.

$$\begin{aligned}
&\text{Distance to 's'} = 0 \\
&\text{Distance to 'a'} = \infty \\
&\text{Distance to 'b'} = \infty \\
&\text{Distance to 'c'} = \infty \\
&\text{Distance to 'd'} = \infty \\
&\text{Distance to 'e'} = \infty \\
&\text{Distance to 'f'} = \infty \\
&\text{Distance to 'g'} = \infty \\
&\text{Distance to 'i'} = \infty
\end{aligned}$$

Iterations

During each iteration, all edges are relaxed.

Iteration 1:

- Relaxing edge 's' to 'a' updates distance to 'a' to 4.
- Relaxing edge 's' to 'c' updates distance to 'c' to -2.
- Relaxing edge 's' to 'f' updates distance to 'f' to 5.
- Continue for all edges...

Iteration 2:

- Continue relaxing each edge...

Negative Cycle Detection

After the final iteration, one additional relaxation step is attempted on each edge to check for negative cycles. If any distance is updated, it indicates a negative cycle.

Final Results

The shortest paths from 's' to each vertex are as follows (assuming no negative cycles):

Vertex	Shortest Distance from 's'
a	4
b	6
c	-2
d	4
e	5
f	0
g	2
i	-4

Iteration	s	a	b	c	d	e	f	g	i
0	0	∞	∞	∞	∞	∞	∞	∞	∞
1	0	4	∞	-2	∞	3	∞	∞	∞
2	0	4	∞	-2	1	2	∞	∞	∞
3	0	3	∞	-2	1	2	∞	∞	∞
4	0	2	∞	-2	1	1	0	∞	∞
5	0	2	4	-2	1	1	0	∞	∞
6	0	2	4	-2	1	1	0	3	∞
7	0	2	4	-2	1	1	0	3	-6
8	0	2	4	-2	-2	1	-1	-3	-5